УЧЕБНИК ПО SVG

Векторная графика для веба

Этот учебник — систематическое введение в SVG (Scalable Vector Graphics) для новичков. SVG — это формат векторной графики, который масштабируется без потерь качества, редактируется текстом, анимируется через CSS и JavaScript, и работает непосредственно в HTML.

Мы начнём с самого начала: что такое вектор, чем он лучше растра, как вставить SVG в HTML. Затем разберём базовые фигуры, контуры <path>, стили, текст, группы, трансформации, анимации, маски, фильтры и доступность. Все примеры можно проверять в браузере — сохраните код в .html файл и откройте двойным щелчком.

ОГЛАВЛЕНИЕ

- Глава 1. Введение в SVG
- Глава 2. Встраивание SVG в HTML
- Глава 3. Структура <svg> и координаты
- Глава 4. Базовые фигуры
- Глава 5. <path> — контуры и кривые
- Глава 6. Стили: заливка и обводка
- Глава 7. Текст в SVG
- Глава 8. Группы, <symbol>, <use>, <defs>
- Глава 9. Transform — трансформации
- Глава 10. Анимации
- Глава 11. Маски, клипы и фильтры
- Глава 12. Доступность, CSS и практика

ГЛАВА 1. ВВЕДЕНИЕ В SVG

1.1. Что такое SVG

SVG (Scalable Vector Graphics — масштабируемая векторная графика) — это формат описания двумерной графики в XML. Каждая фигура, линия, кривая, цвет записаны текстом в виде тегов и атрибутов, точно так же, как в HTML. Браузер читает этот текст и строит по нему картинку.

Простейший SVG выглядит так:

```
<svg xmlns="http://www.w3.org/2000/svg"
     width="100" height="100"
     viewBox="0 0 100 100">
  <circle cx="50" cy="50" r="40"
          fill="#0066cc"/>
</svg>
```

Это синий круг диаметром 80 пикселей в холсте 100×100. В отличие от PNG или JPG, где картинка — это сетка пикселей с заданным цветом каждого, SVG хранит формулу: «круг с центром в (50,50) и радиусом 40, залитый синим». И при масштабировании в 10 раз круг остаётся идеально гладким.

1.2. Вектор vs растр

Растровая графика (PNG, JPG, WebP, GIF) хранит изображение как матрицу пикселей. Каждая точка имеет цвет. Когда вы увеличиваете растровую картинку в 4 раза, браузер дорисовывает промежуточные пиксели интерполяцией — получаются ступеньки и мыло.

Векторная графика (SVG) хранит математическое описание фигур. При увеличении браузер просто перерисовывает круг по формуле с новым радиусом. Картинка всегда остаётся чёткой — на экране Retina, на 4K-мониторе, при печати на баннере 3×5 метров.

Сравнение:

- PNG 200×200 и масштаб x4 — мыло и пиксели
- SVG 200×200 и масштаб x4 — идеально чётко
- PNG логотипа — 50 КБ, на retina мылит
- SVG логотипа — 2 КБ, всегда чёткий
- PNG нельзя изменить без редактора
- SVG — это текст, можно править в блокноте

1.3. Где применяется SVG

SVG идеально подходит для:

- Иконок интерфейса — стрелки, шестерёнки, лупы
- Логотипов и фирменных знаков
- Диаграмм, графиков, инфографики
- Карт: контуры стран, регионов, маршрутов
- Векторных иллюстраций и паттернов
- Анимированных иллюстраций и баннеров
- QR-кодов и штрих-кодов (генерируются)

SVG НЕ подходит для фотографий и сложных многоцветных иллюстраций с плавными градиентами и тенями. Там лучше JPG, WebP или AVIF. SVG работает там, где есть чёткие границы и однородные заливки.

1.4. Преимущества SVG

- Масштабируется без потерь
- Малый размер для графики с малым числом цветов
- Редактируется текстом — diffs в Git работают
- Анимируется через CSS и SMIL
- Доступно для скринридеров через <title>/<desc>
- Индексируется поисковиками (текст в <text>)
- Стилизуется через CSS (fill, stroke, ...)
- Поддерживается всеми браузерами с 2009 года

1.5. Поддержка браузерами

SVG поддерживается во всех современных браузерах: Chrome, Firefox, Safari, Edge, Opera. Даже IE9+ частично поддерживает SVG 1.1. На мобильных — iOS Safari и Android Chrome работают полноценно. Так что можно смело использовать SVG без полифилов и проверок совместимости.

ГЛАВА 2. ВСТРАИВАНИЕ SVG В HTML

2.1. Способ 1: <img>

Самый простой способ — тег <img> с указанием пути к SVG-файлу. Работает как с обычной картинкой:

```
<img src="logo.svg" alt="Логотип"
     width="200" height="60">
```

Плюсы: кэшируется браузером, можно использовать в responsive через srcset. Минусы: нельзя управлять внутренностями SVG через CSS страницы — SVG изолирован. Анимации внутри SVG работают, но hover-эффекты снаружи — нет.

2.2. Способ 2: inline SVG

SVG можно вставить прямо в HTML-код, как любой другой тег. Это даёт максимум возможностей:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Привет, мир!</h1>

    <svg width="100" height="100"
         viewBox="0 0 100 100">
      <circle cx="50" cy="50" r="40"
              fill="#0066cc"/>
    </svg>
  </body>
</html>
```

Плюсы:

- CSS страницы управляет стилями внутри SVG
- JavaScript страницы может манипулировать SVG
- Hover, focus и другие интерактивные эффекты
- Не нужен отдельный файл — всё в одном HTML

Минусы: HTML становится большим, нельзя переиспользовать один SVG на нескольких страницах без копирования. И кэшируется SVG вместе с HTML, а не отдельно.

Inline SVG — стандарт для иконок интерфейсов, анимированных декораций и всего, что должно реагировать на действия пользователя.

2.3. Способ 3: <object>

Тег <object> встраивает внешний SVG, давая промежуточный уровень изоляции и контроля:

```
<object data="logo.svg"
        type="image/svg+xml"
        width="200" height="60">
  Логотип (текст для fallback)
</object>
```

Содержимое тега показывается, если SVG не загрузился. Внутри <object> SVG живёт отдельным документом со своим CSS и JS, но имеет доступ к внешним ресурсам. Этот способ сейчас почти не используется, потому что inline или <img> покрывают большинство сценариев.

2.4. Способ 4: CSS background

SVG можно использовать как фон через CSS:

```
.button {
  background-image: url('icon.svg');
  background-repeat: no-repeat;
  background-position: 8px center;
  background-size: 16px 16px;
  padding-left: 32px;
}
```

Это работает, но SVG полностью изолирован: нельзя поменять цвет через CSS страницы, нельзя hover-эффекты внутри SVG. Для декоративных иконок, которые не должны реагировать на пользователя — нормально.

2.5. Способ 5: <use> + внешний SVG

Современный способ переиспользования — <use> с ссылкой на внешний файл:

```
<!-- icons.svg -->
<svg xmlns="http://www.w3.org/2000/svg"
     style="display:none">
  <symbol id="icon-home" viewBox="0 0 24 24">
    <path d="M12 2 L2 12 H5 V22 H19 V12 H22 Z"/>
  </symbol>
</svg>

<!-- в HTML -->
<svg width="24" height="24">
  <use href="icons.svg#icon-home"/>
</svg>
```

Один файл icons.svg содержит все иконки как <symbol>. На странице иконки вставляются компактно через <use>. Браузер кэширует icons.svg, и каждая иконка на странице — это просто короткая ссылка.

2.6. Какой способ выбрать

Краткая шпаргалка:

- Декоративная картинка без интерактива → <img>
- Иконка с hover/анимацией → inline SVG

- Много иконок на сайте → <use> + внешний спрайт
- Сложная иллюстрация как отдельный документ → <object>
- Декоративный фон → CSS background

ГЛАВА 3. СТРУКТУРА <svg> И КООРДИНАТЫ

3.1. Атрибуты <svg>

Корневой тег <svg> принимает несколько важных атрибутов:

```
<svg xmlns="http://www.w3.org/2000/svg"
     width="200" height="100"
     viewBox="0 0 200 100"
     preserveAspectRatio="xMidYMid meet">
  <!-- содержимое -->
</svg>
```

Разберём каждый:

- xmlns — пространство имён, обязательно для standalone SVG-файлов
- width, height — физический размер в пикселях (или %, em)
- viewBox — логическая система координат (важнейший атрибут)
- preserveAspectRatio — как вписать viewBox в width×height

3.2. Система координат

Координаты SVG считаются от левого верхнего угла. Ось X идёт вправо, ось Y — вниз. Это отличается от математической декартовой системы, где Y идёт вверх. Привыкните — это частая ошибка новичков.

```
<!-- Точка (0, 0) — левый верхний угол -->
<svg width="100" height="100">
  <circle cx="0" cy="0" r="20" fill="red"/>
  <!-- виден только четверть-круг -->
</svg>
```

3.3. viewBox — логические координаты

viewBox — это, пожалуй, самый важный атрибут SVG. Он задаёт «логическую» систему координат, независимую от физических размеров. Синтаксис:

```
viewBox="minX minY width height"
```

Например, viewBox="0 0 100 100" задаёт систему координат от 0 до 100 по обеим осям. Если width="200" height="200", то каждая логическая единица равна 2 физическим пикселям. SVG автоматически масштабирует содержимое.

```
<!-- Логическая система 100×100, -->
<!-- физический размер 200×200: масштаб x2 -->
<svg width="200" height="200"
     viewBox="0 0 100 100">
  <circle cx="50" cy="50" r="40" fill="blue"/>
  <!-- круг с радиусом 80 физических пикселей -->
</svg>
```

Благодаря viewBox вы рисуете в удобных координатах (например, 100×100 или 24×24 для иконок), а физический размер задаёте через width/height.

3.4. Адаптивный SVG

Если убрать width и height, SVG будет занимать всю доступную ширину контейнера, а высота подстроится через viewBox. Это даёт масштабируемость:

```
<svg viewBox="0 0 100 100"
     style="width: 100%; height: auto;">
  <circle cx="50" cy="50" r="40"/>
</svg>
```

Такой SVG растягивается на всю ширину родителя и сохраняет пропорции. Идеально для responsive-дизайна.

3.5. preserveAspectRatio

Если пропорции viewBox и width/height различаются, preserveAspectRatio решает, что делать:

- xMidYMid meet — вписать целиком, центрировать (по умолчанию)
- xMidYMid slice — заполнить, обрезая лишнее
- none — растянуть без сохранения пропорций

```
<svg width="200" height="100"
     viewBox="0 0 100 100"
     preserveAspectRatio="xMidYMid meet">
  <circle cx="50" cy="50" r="40" fill="red"/>
</svg>
<!-- Круг 80×80 вписан в холст 200×100, -->
<-- отцентрирован, по бокам пустые поля -->
```

ГЛАВА 4. БАЗОВЫЕ ФИГУРЫ

4.1. <rect> — прямоугольник

Рисует прямоугольник. Атрибуты:

- x, y — координаты левого верхнего угла
- width, height — размеры
- rx, ry — радиус скругления углов

```
<svg width="200" height="100"
     viewBox="0 0 200 100">
  <rect x="10" y="10" width="80" height="60"
        fill="#0066cc"/>
  <rect x="110" y="10" width="80" height="60"
        rx="10" ry="10"
        fill="#cc6600"/>
</svg>
```

Первый прямоугольник с острыми углами, второй — со скруглёнными (rx, ry задают радиус скругления). Если rx=ry, можно указать только один.

4.2. <circle> — круг

Атрибуты:

- cx, cy — координаты центра
- r — радиус

```
<circle cx="50" cy="50" r="40"
        fill="green"/>
```

Для круга нужно знать центр и радиус. Если вы представляете круг вписанным в прямоугольник, центр вычисляется как (x+width/2, y+height/2).

4.3. <ellipse> — эллипс

Эллипс имеет разные радиусы по горизонтали и вертикали:

- cx, cy — координаты центра
- rx — горизонтальный радиус
- ry — вертикальный радиус

```
<ellipse cx="100" cy="50" rx="80" ry="30"
         fill="#9900cc"/>
```

4.4. <line> — линия

Линия между двумя точками:

- x1, y1 — начальная точка
- x2, y2 — конечная точка

```
<line x1="10" y1="10" x2="90" y2="90"
      stroke="black" stroke-width="2"/>
```

У <line> нет заливки — только обводка (stroke). Без атрибута stroke линия невидима.

4.5. <polyline> — ломаная

Ломаная линия — последовательность отрезков. Точки задаются атрибутом points:

```
<polyline points="10,10 50,80 90,10 130,80"
          fill="none"
          stroke="black" stroke-width="2"/>
```

points — пары x,y через пробел. Полилиния не замыкается автоматически, поэтому fill обычно устанавливают в none. Иначе браузер замкнёт фигуру и зальёт её.

4.6. <polygon> — многоугольник

Многоугольник — как polyline, но автоматически замыкается:

```
<polygon points="50,5 100,90 0,90"
         fill="yellow" stroke="black"/>

<!-- Треугольник -->

<polygon points="50,5 95,90 70,90 50,40 30,90 5,90"
         fill="red"/>

<!-- Шестиконечная звезда -->
```

ГЛАВА 5. <path> — КОНТУРЫ И КРИВЫЕ

5.1. Зачем нужен <path>

Все базовые фигуры (rect, circle, polygon) — это частные случаи контура <path>. <path> умеет рисовать любую форму: от простой линии до сложного логотипа. Фигуры удобны для типовых случаев, path — для всего остального.

Атрибут d (data) содержит последовательность команд. Каждая команда — буква и числа. Заглавная буква — абсолютные координаты, строчная — относительные.

5.2. Команды M и L

M (moveto) — переместить "перо" в точку, не рисуя. L (lineto) — нарисовать линию до точки.

```
<path d="M 10 10 L 90 10 L 90 90 L 10 90 Z"
      fill="none" stroke="black"/>

<!-- Прямоугольник 80×80, -->
<-- начинается в (10,10), идёт в (90,10), -->
<-- затем (90,90), (10,90), Z замыкает -->
```

Z (closepath) — закрыть контур, проведя линию к начальной точке. Без Z контур останется незамкнутым, и fill его не зальёт.

5.3. H и V — горизонталь/вертикаль

H (horizontal lineto) — линия по горизонтали, координата Y не меняется. V (vertical lineto) — по вертикали. Удобно для прямоугольников:

```
<path d="M 10 10 H 90 V 90 H 10 Z"
      fill="none" stroke="black"/>
<!-- Тот же прямоугольник, короче -->
```

5.4. C — кубическая кривая Безье

C (curveto) — кубическая кривая Безье. Самый универсальный тип кривых. Синтаксис: C x1 y1, x2 y2, x y. Две контрольные точки и конечная точка.

```
<path d="M 10 50 C 30 10, 70 10, 90 50"
      fill="none" stroke="black"/>
<!-- S-образная кривая -->
```

Контрольные точки (30,10) и (70,10) «тянут» кривую вверх, создавая сигмоиду. S-команда (smooth curveto) продолжает предыдущую C, автоматически отражая контрольную точку.

5.5. Q — квадратичная кривая Безье

Q (quadratic curveto) — квадратичная кривая Безье. Проще, чем C: одна контрольная точка и конечная. Синтаксис: Q x1 y1, x y.

```
<path d="M 10 50 Q 50 0, 90 50"
      fill="none" stroke="black"/>
<!-- Дуга от (10,50) через (50,0) к (90,50) -->
```

5.6. A — дуга эллипса

A (arc) — дуга эллипса. Самая сложная команда. Синтаксис: A rx ry x-rotation large-arc-flag sweep-flag x y.

- rx, ry — радиусы эллипса
- x-rotation — поворот эллипса в градусах
- large-arc-flag — 0 (малая дуга) или 1 (большая)
- sweep-flag — 0 (против часовой) или 1 (по часовой)
- x, y — конечная точка

```
<path d="M 10 50 A 40 40 0 0 1 90 50"
      fill="none" stroke="black"/>
<!-- Полуокружность от (10,50) до (90,50) -->
```

5.7. Абсолютные и относительные

Заглавные команды (M, L, C) принимают абсолютные координаты от начала системы. Строчные (m, l, c) — относительные от текущей позиции.

```
<path d="M 10 10 l 80 0 l 0 80 l -80 0 Z"
      fill="none" stroke="black"/>
<!-- Прямоугольник 80×80, -->
<-- используя относительные команды -->
```

Здесь первая m ставит перо в (10,10). Дальше l 80 0 — на 80 вправо, l 0 80 — на 80 вниз, l -80 0 — на 80 влево, Z замыкает. Относительные команды удобны, когда фигуру нужно сдвинуть целиком — меняется только первая M.

5.8. Примеры: звезда и сердце

Пятиконечная звезда одним контуром:

```
<path d="M 50 5 L 61 35 L 95 35 L 67 56
         L 78 90 L 50 70 L 22 90 L 33 56
         L 5 35 L 39 35 Z"
      fill="gold" stroke="orange"/>
```

Сердце через две кубические кривые:

```
<path d="M 50 30 C 30 10, 0 30, 50 80
         C 100 30, 70 10, 50 30 Z"
      fill="red"/>
```

ГЛАВА 6. СТИЛИ: ЗАЛИВКА И ОБВОДКА

6.1. fill и stroke

Любую фигуру можно залить (fill) и обвести (stroke). Цвета — любая CSS-форма: имя, HEX, rgb, rgba, hsl.

```
<rect x="10" y="10" width="80" height="60"
      fill="#0066cc"
      stroke="#003366"
      stroke-width="2"/>
```

fill="none" — без заливки. stroke="none" — без обводки. По умолчанию фигуры заливаются чёрным, обводки нет.

6.2. stroke-width

Толщина обводки в логических единицах viewBox. Если viewBox 100×100, а width=200px, stroke-width=2 даст 4 физических пикселя.

6.3. stroke-linecap — концы линий

Как заканчиваются открытые линии:

- butt — обрезаны ровно (по умолчанию)
- round — скруглены
- square — с прямоугольными выступами

```
<line x1="10" y1="10" x2="90" y2="10"
      stroke="black" stroke-width="10"
      stroke-linecap="butt"/>
<line x1="10" y1="30" x2="90" y2="30"
      stroke="black" stroke-width="10"
      stroke-linecap="round"/>
<line x1="10" y1="50" x2="90" y2="50"
      stroke="black" stroke-width="10"
      stroke-linecap="square"/>
```

6.4. stroke-linejoin — углы

Как соединяются сегменты обводки в углах:

- miter — острый угол (по умолчанию)
- round — скруглённый
- bevel — срезанный

6.5. stroke-dasharray — пунктир

Создаёт пунктирную линию. Значение — последовательность длин: длина штриха, длина пропуска, и т.д.

```
<line x1="10" y1="10" x2="190" y2="10"
      stroke="black" stroke-width="2"
      stroke-dasharray="10 5"/>  <!-- штрих 10, пропуск 5 -->

<line x1="10" y1="30" x2="190" y2="30"
      stroke="black" stroke-width="2"
      stroke-dasharray="2 3"/>   <!-- точечный -->

<line x1="10" y1="50" x2="190" y2="50"
      stroke="black" stroke-width="2"
      stroke-dasharray="10 5 2 5"/>  <!-- сложный -->
```

Анимируя stroke-dashoffset через CSS, можно создать эффект «рисующейся» линии (см. главу 10).

6.6. opacity

Прозрачность фигуры целиком через opacity, или по отдельности:

- opacity — прозрачность всей фигуры
- fill-opacity — прозрачность заливки
- stroke-opacity — прозрачность обводки

```
<rect x="10" y="10" width="60" height="60"
      fill="red" opacity="0.5"/>
<rect x="40" y="40" width="60" height="60"
      fill="blue" opacity="0.5"/>
<!-- Полупрозрачные прямоугольники, -->
<-- на пересечении видна смесь цветов -->
```

6.7. <linearGradient>

Линейный градиент: плавный переход между цветами. Определяется в <defs>, применяется через url(#id):

```
<svg width="200" height="100"
     viewBox="0 0 200 100">
  <defs>
    <linearGradient id="grad1"
        x1="0%" y1="0%" x2="100%" y2="0%">
      <stop offset="0%" stop-color="#0066cc"/>
      <stop offset="100%" stop-color="#cc0066"/>
    </linearGradient>
  </defs>
  <rect x="10" y="10" width="180" height="80"
        fill="url(#grad1)"/>
</svg>
```

x1,y1,x2,y2 задают направление градиента (от 0% до 100%). <stop> описывает точки перехода: offset — позиция (0-100%), stop-color — цвет.

6.8. <radialGradient>

Радиальный градиент: переход от центра к краям. Атрибуты cx, cy — центр, r — радиус:

```
<defs>
  <radialGradient id="grad2"
      cx="50%" cy="50%" r="50%">
    <stop offset="0%" stop-color="yellow"/>
    <stop offset="100%" stop-color="red"/>
  </radialGradient>
</defs>
<circle cx="50" cy="50" r="40"
        fill="url(#grad2)"/>
```

6.9. <pattern> — узор заливки

Pattern позволяет заливать фигуры повторяющимся узором. Определяется тоже в <defs>:

```
<defs>
  <pattern id="dots"
          width="20" height="20"
          patternUnits="userSpaceOnUse">
    <circle cx="10" cy="10" r="3" fill="red"/>
  </pattern>
</defs>
<rect x="0" y="0" width="200" height="100"
      fill="url(#dots)"/>
```

ГЛАВА 7. ТЕКСТ В SVG

7.1. <text>

Тег <text> рисует текст в заданной точке. Базовые атрибуты:

- x, y — координаты базовой линии
- font-family, font-size, font-weight
- fill — цвет текста
- text-anchor — выравнивание относительно (x,y)
- dominant-baseline — вертикальное выравнивание

```
<text x="50" y="50"
      font-family="Arial" font-size="24"
      fill="black"
      text-anchor="middle">Привет!</text>
```

7.2. text-anchor

text-anchor определяет, как текст выравнивается относительно точки (x, y):

- start — текст идёт вправо от точки (по умолчанию)
- middle — текст центрирован относительно точки
- end — текст заканчивается в точке

```
<text x="100" y="30" text-anchor="start">Слева</text>
<text x="100" y="60" text-anchor="middle">Центр</text>
<text x="100" y="90" text-anchor="end">Справа</text>
```

7.3. dominant-baseline

dominant-baseline — вертикальное выравнивание:

- alphabetic — по базовой линии (по умолчанию)
- middle — по середине текста
- hanging — по верхней линии
- central — по центру между ascent/descent

```
<rect x="0" y="0" width="100" height="40"
      fill="#eee"/>
<text x="50" y="20"
      text-anchor="middle"
      dominant-baseline="middle">В центре</text>
```

7.4. <tspan> — подстроки

Внутри <text> можно использовать <tspan> для частей текста с разными стилями:

```
<text x="10" y="50" font-size="20">
  Цена: <tspan font-weight="bold"
            fill="red">999 ₽</tspan>
  (со скидкой)
</text>
```

<tspan> может иметь свои координаты x, y, что позволяет переносить слова на новые строки вручную:

```
<text font-size="16">
  <tspan x="10" y="20">Первая строка</tspan>
  <tspan x="10" y="40">Вторая строка</tspan>
</text>
```

7.5. <textPath> — текст вдоль пути

Уникальная возможность SVG: текст может идти вдоль произвольной кривой. Сначала определяется <path> в <defs>, потом на него ссылается <textPath>:

```
<defs>
  <path id="curve" d="M 10 90 Q 100 0, 190 90"/>
</defs>

<text font-size="18">
  <textPath href="#curve">
    Текст идёт вдоль дуги
  </textPath>
</text>
```

ГЛАВА 8. ГРУППЫ, <symbol>, <use>, <defs>

8.1. <g> — группировка

Тег <g> объединяет несколько элементов в группу. Атрибуты, заданные на <g>, наследуются детьми:

```
<g fill="red" stroke="black" stroke-width="2">
  <circle cx="30" cy="30" r="20"/>
  <circle cx="70" cy="30" r="20"/>
  <circle cx="50" cy="60" r="20"/>
</g>
<!-- Все круги красные с чёрной обводкой -->
```

Группы можно трансформировать (transform на <g>) и стилизовать через CSS как единое целое.

8.2. <defs> — определения

<defs> хранит элементы, которые будут переиспользованы. Содержимое <defs> НЕ отображается напрямую, только через <use> или url(#id):

```
<defs>
  <linearGradient id="g1">...</linearGradient>
  <pattern id="dots">...</pattern>
  <symbol id="heart" viewBox="0 0 24 24">
    <path d="M12 21s-7-4.5-9-9 ..."/>
  </symbol>
</defs>
```

8.3. <symbol> — шаблон

<symbol> — это почти как <g>, но предназначен именно для переиспользования через <use>. В отличие от <g>, <symbol> не отображается сам по себе, даже если не в <defs>. У <symbol> есть собственный viewBox, что позволяет растягивать его в любых пропорциях при вставке:

```
<svg style="display:none">
  <symbol id="icon-home" viewBox="0 0 24 24">
    <path d="M12 2 L2 12 L4 12 L4 22 H10 V14 H14 V22 H20 V12 L22 12 Z"/>
  </symbol>
</svg>

<svg width="24" height="24" fill="currentColor">
  <use href="#icon-home"/>
</svg>
```

8.4. <use> — вставка

<use> создаёт экземпляр элемента по ссылке. Поддерживает x, y, width, height для позиционирования и масштабирования:

```
<use href="#icon-home"
     x="10" y="10"
     width="24" height="24"/>
```

8.5. SVG-спрайты

На основе <symbol> + <use> строятся SVG-спрайты — наборы иконок в одном файле. Преимущества перед отдельными файлами: один HTTP-запрос, кэширование всего набора, лёгкое переиспользование.

```
<!-- icons.svg, скрытый -->
<svg xmlns="http://www.w3.org/2000/svg"
     style="display:none">
  <symbol id="i-home" viewBox="0 0 24 24">
    <path d="M12 2 L2 12 ..."/>
  </symbol>
  <symbol id="i-user" viewBox="0 0 24 24">
    <path d="M12 12 a5 5 0 1 0 ..."/>
  </symbol>
</svg>

<!-- в HTML -->
<svg width="24" height="24"><use href="icons.svg#i-home"/></svg>
<svg width="24" height="24"><use href="icons.svg#i-user"/></svg>
```

8.6. currentColor

Особенность SVG: можно задать fill="currentColor" и тогда цвет будет наследоваться от CSS color родительского элемента. Это делает иконки «тематическими» — один и тот же SVG меняет цвет в зависимости от контекста:

```
<style>
  .btn-primary { color: blue; }
  .btn-danger  { color: red; }
</style>

<button class="btn-primary">
  <svg width="16" height="16">
    <use href="#icon-home"/>
  </svg>
  Главная
</button>

<!-- В icons.svg -->
<symbol id="icon-home">
  <path fill="currentColor" d="..."/>
</symbol>
```

ГЛАВА 9. TRANSFORM — ТРАНСФОРМАЦИИ

9.1. Атрибут transform

Любому элементу SVG можно задать атрибут transform с одной или несколькими функциями:

```
<rect x="10" y="10" width="40" height="40"
      transform="translate(50, 50) rotate(45)"
      fill="red"/>
```

Здесь прямоугольник сначала сдвигается на (50, 50), потом поворачивается на 45 градусов. Порядок важен: трансформации применяются справа налево, как умножение матриц.

9.2. translate(x, y)

Сдвиг на указанные координаты:

```
<circle cx="0" cy="0" r="10"
        transform="translate(50, 50)"/>
```

Круг с центром в (0,0) после transform будет отрисован в (50,50).

9.3. scale(sx, sy)

Масштабирование. Если sy не указан, используется sx:

```
<rect x="0" y="0" width="10" height="10"
      transform="scale(2)" fill="red"/>
<!-- Прямоугольник 20×20 -->

<rect x="0" y="0" width="10" height="10"
      transform="scale(2, 1)" fill="blue"/>
<-- Прямоугольник 20×10 (растянут по X) -->
```

Важно: scale масштабирует не только размер, но и координаты. Если у прямоугольника x=10, то после scale(2) он будет в x=20. Поэтому обычно рисуют в (0,0) и передвигают через translate.

9.4. rotate(angle, cx, cy)

Поворот на angle градусов вокруг точки (cx, cy). Без cx,cy — вокруг (0,0):

```
<rect x="-20" y="-10" width="40" height="20"
      transform="rotate(45)"
      fill="red"/>
<-- Прямоугольник повёрнут на 45° вокруг (0,0) -->

<rect x="30" y="40" width="40" height="20"
      transform="rotate(45, 50, 50)"
      fill="blue"/>
<-- Вокруг центра (50, 50) -->
```

9.5. skewX и skewY

Наклон по горизонтали (skewX) или вертикали (skewY). Аргумент — угол в градусах:

```
<rect x="10" y="10" width="40" height="40"
      transform="skewX(20)"
      fill="red"/>
<-- Прямоугольник наклонён по горизонтали -->
```

9.6. matrix(a,b,c,d,e,f)

Полная аффинная трансформация через матрицу 2×3. Все остальные функции — это частные случаи matrix. Чаще используется внутренне браузером или графическими редакторами при экспорте SVG.

9.7. transform через CSS

В современном SVG трансформации можно задавать через CSS, что позволяет анимировать их:

```
<style>
  .box {
    transform-origin: 50px 50px;
    transform: rotate(0deg);
    transition: transform 0.3s;
  }
  .box:hover {
    transform: rotate(45deg);
  }
</style>

<rect class="box" x="30" y="30" width="40" height="40"/>
<-- Поворачивается на 45° при наведении -->
```

transform-origin задаёт точку поворота. По умолчанию для SVG это (0, 0) — левый верхний угол, в отличие от HTML, где по умолчанию центр элемента. Это частая ловушка новичков.

ГЛАВА 10. АНИМАЦИИ

10.1. CSS-анимации

Inline SVG можно анимировать через обычный CSS, как любой другой HTML-элемент:

```
<style>
  .pulse {
    animation: pulse 2s ease-in-out infinite;
    transform-origin: center;
  }
  @keyframes pulse {
    0%, 100% { transform: scale(1); }
    50%      { transform: scale(1.1); }
  }
</style>

<svg width="100" height="100" viewBox="0 0 100 100">
  <circle class="pulse" cx="50" cy="50" r="30"
          fill="#0066cc"/>
</svg>
```

Можно анимировать большинство свойств: fill, stroke, stroke-width, opacity, transform. Не анимируются атрибуты вида cx, cy, r, x, y — это свойства элементов, а не CSS-стили.

10.2. CSS transitions на hover

Простейший интерактив — hover-эффект:

```
<style>
  .icon { transition: fill 0.3s; fill: gray; }
  .icon:hover { fill: red; }
</style>

<svg width="24" height="24" viewBox="0 0 24 24">
  <path class="icon" d="M12 21s-7-4.5-9-9 ..."/>
</svg>
```

Иконка становится красной при наведении. Это самый частый паттерн для иконок интерфейса.

10.3. SMIL: <animate>

SVG имеет собственный язык анимаций — SMIL. Самый простой тег — <animate>, который встраивается внутрь анимируемого элемента:

```
<circle cx="50" cy="50" r="20" fill="red">
  <animate attributeName="r"
           from="20" to="40"
           dur="2s"
           repeatCount="indefinite"
           values="20;40;20"/>
</circle>
```

Здесь круг пульсирует: радиус меняется от 20 до 40 и обратно. Атрибут values задаёт ключевые кадры через точку с запятой.

10.4. SMIL: <animateTransform>

<animateTransform> анимирует transform. Атрибут type указывает, какую трансформацию анимировать:

```
<rect x="-20" y="-10" width="40" height="20"
      fill="red">
  <animateTransform attributeName="transform"
                    type="rotate"
                    from="0" to="360"
                    dur="4s"
                    repeatCount="indefinite"/>
</rect>
```

Прямоугольник вращается вокруг (0,0) с периодом 4 секунды.

10.5. SMIL vs CSS vs JS

Сегодня SMIL считается устаревшим, особенно после того как Chrome планировал его убрать (потом планы отменили). Современный подход: CSS-анимации для большинства случаев, JS (через requestAnimationFrame) для сложных сценариев. SMIL по-прежнему работает и удобен для автономных SVG-файлов без CSS.

10.6. Эффект «рисующейся» линии

Частый приём — анимация stroke-dashoffset для создания иллюзии рисования линии:

```
<style>
  .draw {
    stroke-dasharray: 1000;
    stroke-dashoffset: 1000;
    animation: draw 3s ease-out forwards;
  }
  @keyframes draw {
    to { stroke-dashoffset: 0; }
  }
</style>

<svg width="200" height="100" viewBox="0 0 200 100">
  <path class="draw" d="M 10 50 Q 100 0, 190 50"
        fill="none" stroke="black" stroke-width="3"/>
</svg>
```

stroke-dasharray: 1000 создаёт один длинный штрих, stroke-dashoffset сдвигает его за пределы видимости. Анимация уменьшает offset до 0, и линия «прорисовывается».

ГЛАВА 11. МАСКИ, КЛИПЫ И ФИЛЬТРЫ

11.1. <clipPath> — обрезка

ClipPath обрезает фигуру по контуру. То, что попало внутрь контура — видно, что не попало — скрыто:

```
<defs>
  <clipPath id="circle-clip">
    <circle cx="50" cy="50" r="40"/>
  </clipPath>
</defs>

<image href="photo.jpg"
       x="0" y="0" width="100" height="100"
       clip-path="url(#circle-clip)"/>
```

Фотография будет видна только внутри круга. Применяется для аватарок, портретов в круглых рамках и т.д.

11.2. <mask> — маска

Маска похожа на clipPath, но работает через альфа-канал. Белые области маски полностью видимы, чёрные — невидимы, серые — полупрозрачны:

```
<defs>
  <mask id="fade-mask">
    <linearGradient id="fade-grad"
        x1="0" y1="0" x2="0" y2="1">
      <stop offset="0%" stop-color="white"/>
      <stop offset="100%" stop-color="black"/>
    </linearGradient>
    <rect x="0" y="0" width="100" height="100"
          fill="url(#fade-grad)"/>
  </mask>
</defs>

<image href="photo.jpg"
       width="100" height="100"
       mask="url(#fade-mask)"/>
```

Фотография плавно исчезает сверху вниз. Это недоступно через clipPath, который даёт только жёсткие границы.

11.3. <filter> — фильтры

Фильтры применяют графические эффекты: размытие, тень, обесцвечивание и т.д. Фильтр состоит из примитивов, которые соединяются в цепочку:

```
<defs>
  <filter id="shadow">
    <feGaussianBlur in="SourceAlpha"
                    stdDeviation="3"/>
    <feOffset dx="2" dy="2"/>
    <feMerge>
      <feMergeNode/>
      <feMergeNode in="SourceGraphic"/>
    </feMerge>
  </filter>
</defs>

<rect x="10" y="10" width="80" height="60"
      fill="red" filter="url(#shadow)"/>
```

Здесь: feGaussianBlur размывает альфу (создавая силуэт), feOffset сдвигает тень, feMerge накладывает тень под□□. Результат — прямоугольник с тенью.

11.4. feDropShadow — простая тень

В современных браузерах есть короткий путь — feDropShadow:

```
<filter id="shadow2">
  <feDropShadow dx="2" dy="2"
                stdDeviation="3"
                flood-color="black"
                flood-opacity="0.5"/>
</filter>
```

А ещё в CSS есть filter: drop-shadow(...), который применяется к элементу напрямую:

```
<style>
  .icon { filter: drop-shadow(2px 2px 3px rgba(0,0,0,0.5)); }
</style>
```

11.5. feGaussianBlur — размытие

Размытие по Гауссу полезно для фоновых эффектов:

```
<filter id="blur">
  <feGaussianBlur stdDeviation="5"/>
</filter>

<image href="bg.jpg" filter="url(#blur)"/>
```

11.6. feColorMatrix — обесцвечивание

feColorMatrix позволяет манипулировать цветами через матрицу 4×5. Обесцвечивание (grayscale):

```
<filter id="gray">
  <feColorMatrix type="matrix"
    values="0.3 0.6 0.1 0 0
            0.3 0.6 0.1 0 0
            0.3 0.6 0.1 0 0
            0   0   0   1 0"/>
</filter>
```

ГЛАВА 12. ДОСТУПНОСТЬ, CSS И ПРАКТИКА

12.1. <title> и <desc>

Для доступности каждая значимая SVG-картинка должна иметь <title> (заголовок) и <desc> (описание). Скринридеры их читают:

```
<svg role="img"
     aria-labelledby="title desc"
     viewBox="0 0 100 100" width="100" height="100">
  <title id="title">Логотип компании</title>
  <desc id="desc">Синий круг с белой буквой A в центре</desc>
  <circle cx="50" cy="50" r="40" fill="#0066cc"/>
  <text x="50" y="55" text-anchor="middle"
        fill="white" font-size="30">A</text>
</svg>
```

12.2. aria-label для декоративных SVG

Декоративные SVG, не несущие смысла, нужно скрывать от скринридеров:

```
<svg aria-hidden="true"
     viewBox="0 0 24 24" width="24" height="24">
  <path d="..."/>
</svg>
```

Для интерактивных SVG (кнопок, ссылок с иконкой) — добавляйте aria-label на родительский элемент с описанием действия:

```
<button aria-label="Закрыть">
  <svg aria-hidden="true" width="16" height="16">
    <path d="M0 0 L16 16 M16 0 L0 16" stroke="black"/>
  </svg>
</button>
```

12.3. Стилизация через CSS

Inline SVG можно стилизовать через CSS, как любой HTML. Можно использовать class, id, атрибуты. CSS-переменные работают:

```
<style>
  .icon { fill: var(--icon-color, gray); }
  .icon:hover { fill: var(--icon-hover, blue); }
</style>

<svg class="icon" width="24" height="24">
  <use href="#icon-home"/>
</svg>
```

12.4. Задача 1: иконка бургер-меню

Создайте иконку бургер-меню: три горизонтальные линии. Решение:

```
<svg width="24" height="24"
     viewBox="0 0 24 24"
     fill="none" stroke="currentColor"
     stroke-width="2"
     stroke-linecap="round">
  <line x1="3" y1="6"  x2="21" y2="6"/>
  <line x1="3" y1="12" x2="21" y2="12"/>
  <line x1="3" y1="18" x2="21" y2="18"/>
</svg>
```

12.5. Задача 2: логотип-круг с буквой

Круг с буквой внутри, с hover-эффектом. Решение:

```
<style>
  .logo { transition: transform 0.3s; cursor: pointer; }
  .logo:hover { transform: rotate(15deg); }
</style>

<svg class="logo" width="80" height="80"
     viewBox="0 0 80 80">
  <circle cx="40" cy="40" r="35" fill="#0066cc"/>
  <text x="40" y="48" text-anchor="middle"
        fill="white" font-family="Arial"
        font-size="32" font-weight="bold">A</text>
</svg>
```

12.6. Задача 3: кнопка с анимированной иконкой

Кнопка с иконкой дома, при наведении иконка подпрыгивает. Решение:

```
<style>
  .btn-home { background: #0066cc; color: white;
    border: none; padding: 8px 16px;
    border-radius: 4px; cursor: pointer;
    display: inline-flex; align-items: center;
    gap: 8px;
  }
  .btn-home svg { transition: transform 0.2s; }
  .btn-home:hover svg { transform: translateY(-3px); }
</style>

<button class="btn-home" aria-label="На главную">
  <svg width="16" height="16" viewBox="0 0 24 24"
       fill="currentColor" aria-hidden="true">
    <path d="M12 2 L2 12 L4 12 L4 22 H10 V14 H14 V22 H20 V12 L22 12 Z"/>
  </svg>
  Главная
</button>
```

12.7. Задача 4: диаграмма-пончик

Круговая диаграмма в виде бублика (donut). Решение через stroke-dasharray:

```
<svg width="100" height="100" viewBox="0 0 100 100">
  <!-- Фон: серый круг -->
  <circle cx="50" cy="50" r="40"
          fill="none" stroke="#eee" stroke-width="12"/>

  <!-- 70% прогресса: синяя дуга -->
  <!-- Длина окружности = 2*PI*40 = ~251 -->
  <!-- 70% от 251 = ~176 -->
  <circle cx="50" cy="50" r="40"
          fill="none" stroke="#0066cc" stroke-width="12"
          stroke-dasharray="176 251"
          stroke-dashoffset="63"
          transform="rotate(-90 50 50)"/>

  <text x="50" y="55" text-anchor="middle"
        font-size="20" font-weight="bold">70%</text>
</svg>
```

Трюк в том, что длина окружности круга с r=40 равна 2π·40 ≈ 251. stroke-dasharray="176 251" делает штрих длиной 176 (70% от 251) и пропуск 251. stroke-dashoffset=63 сдвигает старт дуги. rotate(-90) поворачивает старт на верх круга.

12.8. Задача 5: фоновый паттерн

Создайте фоновый паттерн из точек. Решение:

```
<svg width="100%" height="100%">
  <defs>
    <pattern id="dots"
             width="20" height="20"
             patternUnits="userSpaceOnUse">
      <circle cx="10" cy="10" r="2"
              fill="#0066cc" fill-opacity="0.3"/>
    </pattern>
  </defs>
  <rect width="100%" height="100%"
        fill="url(#dots)"/>
</svg>
```

12.9. Задача 6: спрайт иконок

Создайте SVG-спрайт с тремя иконками: дом, пользователь, шестерёнка. Используйте на странице. Решение:

```
<!-- icons.svg -->
<svg xmlns="http://www.w3.org/2000/svg"
     style="display:none">
  <symbol id="i-home" viewBox="0 0 24 24">
    <path fill="currentColor"
          d="M12 2 L2 12 L4 12 L4 22 H10 V14 H14 V22 H20 V12 L22 12 Z"/>
  </symbol>
  <symbol id="i-user" viewBox="0 0 24 24">
    <path fill="currentColor"
          d="M12 12 a5 5 0 1 0 0 -10 a5 5 0 1 0 0 10
             M4 22 a8 8 0 0 1 16 0"/>
  </symbol>
  <symbol id="i-cog" viewBox="0 0 24 24">
    <path fill="currentColor"
          d="M12 8 a4 4 0 1 0 0 8 a4 4 0 1 0 0 -8
             M12 2 v3 M12 19 v3
             M2 12 h3 M19 12 h3" stroke="currentColor"
          stroke-width="2"/>
  </symbol>
</svg>

<!-- в HTML -->
<nav style="display:flex; gap:16px">
  <a href="/" aria-label="Главная">
    <svg width="24" height="24" color="#0066cc">
      <use href="icons.svg#i-home"/>
    </svg>
  </a>
  <a href="/profile" aria-label="Профиль">
    <svg width="24" height="24" color="#0066cc">
      <use href="icons.svg#i-user"/>
    </svg>
  </a>
  <a href="/settings" aria-label="Настройки">
    <svg width="24" height="24" color="#0066cc">
      <use href="icons.svg#i-cog"/>
    </svg>
  </a>
</nav>
```

12.10. Задача 7: hover-карточка с волной

Карточка товара с анимированной волной при наведении. Решение:

```
<style>
  .card {
    position: relative;
    width: 200px; padding: 16px;
    border: 1px solid #ddd; border-radius: 8px;
    overflow: hidden;
  }
  .card .wave {
    position: absolute; bottom: -20px;
    left: -50%; width: 200%; height: 30px;
    transition: transform 0.4s ease;
  }
  .card:hover .wave { transform: translateY(-15px); }
</style>

<div class="card">
  <h3>Товар</h3>
  <p>Описание товара...</p>
  <svg class="wave" viewBox="0 0 200 30"
       preserveAspectRatio="none" aria-hidden="true">
    <path d="M0 15 Q 50 0, 100 15 T 200 15 V30 H0 Z"
          fill="#0066cc"/>
  </svg>
</div>
```

12.11. Итоги по главам

Глава 1. SVG — векторный формат XML. Масштабируется без потерь, мал для графики с малым числом цветов, редактируется текстом. Идеален для иконок, логотипов, диаграмм.

Глава 2. Пять способов встраивания: <img>, inline, <object>, CSS background, <use> + внешний спрайт. Inline даёт максимум контроля через CSS/JS.

Глава 3. Атрибуты <svg>: xmlns, width, height, viewBox (логические координаты), preserveAspectRatio. Координаты: (0,0) — левый верхний угол, Y — вниз.

Глава 4. Базовые фигуры: <rect>, <circle>, <ellipse>, <line>, <polyline>, <polygon>. Каждая имеет свой набор атрибутов.

Глава 5. <path d=...> — универсальный контур. Команды M/L/H/V/C/Q/A/Z. Заглавные — абсолютные, строчные — относительные. Z — закрыть контур.

Глава 6. fill, stroke, stroke-width, stroke-linecap/linejoin, stroke-dasharray. <linearGradient>, <radialGradient>, <pattern> в <defs>. opacity и fill-opacity/stroke-opacity.

Глава 7. <text> с x,y, font-family/size/weight, text-anchor (start/middle/end), dominant-baseline. <tspan> для частей. <textPath> — текст вдоль кривой.

Глава 8. <g> — группировка с наследованием атрибутов. <defs> — определения. <symbol> — шаблон. <use> — вставка. SVG-спрайты через <symbol> + <use>. currentColor для тематических иконок.

Глава 9. transform: translate, scale, rotate, skewX/Y, matrix. Порядок важен. CSS transform с transform-origin — для анимаций.

Глава 10. CSS-анимации и transitions — современный стандарт. SMIL <animate> и <animateTransform> — для автономных SVG. stroke-dashoffset для эффекта «рисующейся» линии.

Глава 11. <clipPath> — жёсткая обрезка, <mask> — через альфа-канал. <filter> с примитивами: feGaussianBlur, feOffset, feColorMatrix, feDropShadow, feMerge. CSS filter: drop-shadow как короткий путь.

Глава 12. Доступность: <title>, <desc>, role="img", aria-label, aria-hidden. CSS-стилизация через class и CSS-переменные. Семь практических задач: бургер-меню, логотип, кнопка с анимацией, диаграмма-пончик, паттерн, спрайт иконок, hover-карточка.

12.12. Что читать дальше

После освоения этого учебника логично двигаться в трёх направлениях. Во-первых, CSS-анимации и transition — основа современного веб-движения. Изучите @keyframes, cubic-bezier, transform, animation-timing-function.

Во-вторых, инструменты: Figma и Illustrator для создания SVG, SVGO для оптимизации (удаляет метаданные, лишние точки, упрощает пути). svgr для конвертации SVG в React-компоненты.

В-третьих, библиотеки: D3.js для сложных визуализаций, Snap.svg и SVG.js для программной манипуляции, Lottie для воспроизведения After Effects анимаций через SVG/Canvas.

Главный совет: не пытайтесь заучить все команды path наизусть. SVG — это практика. Нарисуйте свой логотип, сделайте иконки для навигации, постройте диаграмму для своего проекта. С каждой новой фигурой приходит интуитивное понимание, когда нужен path, а когда polygon, когда градиент, а когда pattern. Опыт складывается из десятков нарисованных и переделанных картинок, а не из прочитанных справочников.
